

- 1 Az Agentic AI Fejlesztés Kihívásai és Illúziói a Hagyományos Nagyvállalati Környezetben
 - 1.1 Bevezető: A Ferrari motor a lovaskocsiban
 - 1.2 1. Infrastrukturális béklyók és a Fragmentált Architektúra
 - 1.2.1 Elavult fejlesztői környezetek (A Linux és Python/TypeScript dominanciája)
 - 1.2.2 Technikai adósság és legacy integrációk
 - 1.3 2. A Fejlesztői Szerepkörök és az “AI Vibe Coding” Válsága
 - 1.4 3. A Hagyományos Csapatstruktúra Felbomlása és az Új Együttműködési Modell
 - 1.4.1 A “Magányos Farkasok” kora, vagy inkább “Szuper-csapatok”?
 - 1.4.2 A Frontend “Vakfolt”: A vizuális visszatesztelés nehézségei
 - 1.4.3 Interakció és Cross-Review az SDD (Spec-Driven Development) ciklusban
 - 1.5 4. A Strukturált Szoftverfejlesztési Életciklus (SDLC) és a CI/CD Folyamatok Anakronizmusa
 - 1.5.1 A hagyományos “Tervezünk-Kódolunk-Review-zunk” láncolat lassúsága
 - 1.5.2 A manuális tesztelés megdöbbenő túlélése
 - 1.5.3 A teszt-automatizáció félreértése
 - 1.5.4 Környezeteken való ugrálás (Environment Jumping)
 - 1.5.5 Az Új AI-vezérelt CI/CD Pipeline és a Környezetek Sűrítése
 - 1.6 5. Adatvédelem, Adatbiztonság és Szabályozási Megfelelőség (Compliance)
 - 1.6.1 Szigorú előírások (DORA, GDPR és MNB)
 - 1.7 6. Jogosultságkezelés és Központosított Hitelesítés (IAM)
 - 1.8 7. Ágens-specifikus Biztonsági Kockázatok (Prompt Injection és Tool Misuse)
 - 1.9 8. Költségmenedzsment és Token-ökönómia (Token Economics)
 - 1.10 9. Fokozatos Autonómia (Progressive Autonomy)
 - 1.11 10. Observability (Megfigyelhetőség), Telemetria és AgentOps
 - 1.12 11. Az Üzleti Döntéshozatal Bürokratikus Szűk Keresztmetszete és Szervezeti Változáskezelés
 - 1.12.1 Szigorú AI Workflow-k és a “Harness Engineering”
 - 1.12.2 Dedikált idő a projekt-specifikus finomhangolásra (Improvement Loops)
 - 1.12.3 Belső AI Center of Excellence (CoE)
 - 1.13 12. A Vállalati Tudásbázis Állapota (A “Garbage In, Garbage Out” probléma)
 - 1.14 13. Jogi Felelősség (Accountability & Liability)
 - 1.15 14. Beszállítói Kitétség (Vendor Lock-in) és Modell-Agnosztikusság

1 Az Agentic AI Fejlesztés Kihívásai és Illúziói a Hagyományos Nagyvállalati Környezetben

1.1 Bevezető: A Ferrari motor a lovaskocsiban

A modern nagyvállalati (különösen banki) szektorban az Agentic AI bevezetése gyakran egy alapvető félreértésen alapul: a vezetőség abban a hitben él, hogy az AI eszközök pusztán integrálásával a fejlesztési ciklusok azonnal, radikálisan lerövidülnek. A valóságban azonban egy transzformálatlan, bürokratikus szervezetre ráhúzni az autonóm AI fejlesztést kontraproduktív. Hiába képes egy ágens megírni a kódot másodpercek alatt, ha a technikai adósság, a silósított szervezeti felépítés és az “emberi sebességre” (human speed) optimalizált folyamatok hónapokra megakasztják a tényleges szállítást.

Az alábbi dokumentum összefoglalja azokat a kritikus infrastrukturális, architektúráis, szabályozási és szervezeti szűk keresztmetszeteket, amelyek megakadályozzák az Agentic AI sikeres enterprise skálázását a szigorúan szabályozott pénzügyi szektorban.

1.2 1. Infrastrukturális béklyók és a Fragmentált Architektúra

1.2.1 Elavult fejlesztői környezetek (A Linux és Python/TypeScript dominanciája)

Az Agentic AI ökoszisztéma agilitást és specifikus technológiai alapot követel. Az iparág *de facto* standardja a **Linux operációs rendszer**, a **Python**, valamint a **TypeScript**. * **Miért Linux?** Az ágensek autonóm futtatásához és izolált végrehajtásához biztonságos, konténerizált (Docker/Sandbox) környezetekre van szükség. Továbbá a natív AI keretrendszerek és a GPU-gyorsítások (pl. NVIDIA CUDA/cuDNN) alapértelmezetten Linuxra vannak optimalizálva. A bankokban megszokott, zárt Windows munkaállomásokon ezek a modern AI toolok natívan nem, vagy csak komoly kompromisszumokkal (pl. WSL2) futnak. * **Miért Python vagy TypeScript?** A modern ágens-keretrendszerek (pl. LangChain, Vercel AI SDK, AutoGen) szinte kizárólag ezen a két nyelven kapnak azonnali (“day-zero”) támogatást. A Python a háttérbeli gondolkodási folyamatok és az adatmanipuláció vitathatatlan iparági standardja, míg a TypeScript a webes felületekkel és böngészős eszközökkel integrált ágensek alapja.

Ezzel szemben az enterprise banki architektúra gyakran nehézkes, monolitikus **Java backendekre** és Windows rendszerekre épül. Míg a Java kiváló az operációs fegyelem és a tranzakciós rendszerek terén, a túlzottan verbózus és statikusan típusos nyelvezet rendkívül merev az AI gyors prototipizálásához, ráadásul hiányzik belőle a Python natív adatmanipulációs (Pandas, TensorFlow) ökoszisztémája. Egy kizárólag Windowsra és Javára optimalizált környezetben az AI ágensek adta sebességelőny egyszerűen elvész az integrációs súrlódások (pl. WSL2 virtualizációs réteg kényszerű használata) miatt.

1.2.2 Technikai adósság és legacy integrációk

Az ágensek (akik már nem csak olvasnak, de cselekszenek is) akkor működnek jól, ha tiszta, szabványosított API-okon keresztül tudnak integrálódni. Ha egy környezetben egyedi, töredezett legacy backend megoldások futnak, az AI egyszerűen elakad a technikai adósságban. A sikeres autonóm működés megköveteli a központosított architektúrát, amely felváltja az egyedi implementációs káoszt – például a legacy rendszerek és egyedi API-k (mint a banki PSD2/TPP megoldások) kiváltását egy egységes, standardizált integrációs réteggel –, és egységes, kiszámítható végpontokat biztosít a rendszerek számára.

1.3 2. A Fejlesztői Szerepkörök és az “AI Vibe Coding” Válsága

Forrás: *Exploring Gen AI*

A nagyvállalatoknál a fejlesztői csapatok jellemzően szét vannak tagolva: az üzleti elemzők és архитекtek látják át a rendszert és a komponensek egymásra hatását, míg a programozók pusztán implementálják a specifikációt.

Az AI licenceket jelenleg jellemzően az utóbbi rétegnek adják, aminek az eredménye a “vibe coding”: az AI-t a rendszer egészét nem látó fejlesztők használják, és a hagyományos emberi kódolást próbálják beleerőszakolni az agentic módszertanba. A paradigmaváltás ott kezdődik, ha az AI-t a tervezői/architekti réteg kapja meg, akik a szakmai tudásukkal végig tudják vinni az agentic fejlesztést, és az AI segítségével képesek a rutinszerű, mechanikus implementációért felelős operatív kódolási réteg munkáját kiváltani vagy jelentősen felgyorsítani.

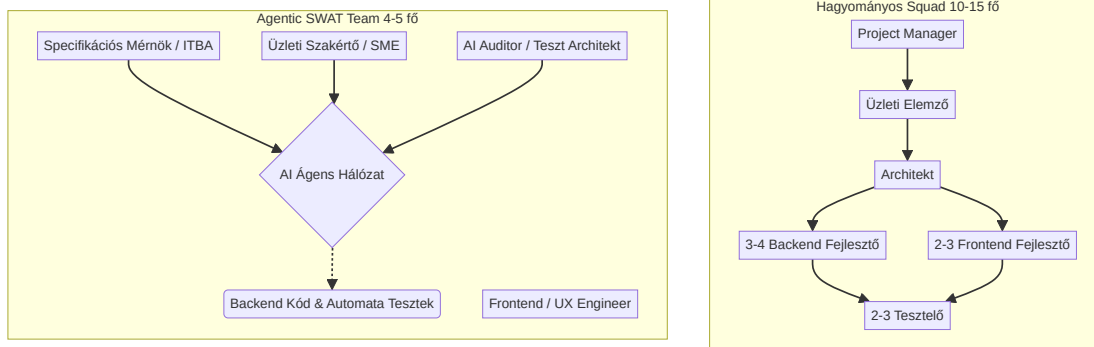
1.4 3. A Hagyományos Csapatstruktúra Felbomlása és az Új Együttműködési Modell

Forrás: *IBM: Agile and AI Productivity*

Jelenleg egy tipikus banki projektcsapat (squad) 10-15 főből áll: üzleti elemző (ITBA), архитекtek, több dedikált frontend és backend fejlesztő, manuális tesztelők és tesztautomatizálók. Agentic AI módban ez a nehézkes, többszereplős modell fenntarthatatlanná válik.

1.4.1 A “Magányos Farkasok” kora, vagy inkább “Szuper-csapatok”?

Az új paradigmában eljön a “magányos farkasok” (pontosabban: az egy személyben teljes rendszereket átlátó mérnökök) kora. Egyetlen szenior Architect vagy ITBA képes egy teljes microservice végponttól-végpontig (frontendtől az adatbázisig) történő lefejlesztését végigvinni egy ágens segítségével. Nem a kódolók száma számít, hanem a rendszer absztrakciós megértése. A korábbi 10-15 fős csapat helyett egy 4-5 fős, magasan kvalifikált “SWAT csapat” is elegendő:



- 1. Specifikációs Mérnök / ITBA (System Architect):** Ő felel az üzleti igények strukturált leírásáért. Az interaktív specifikáció-menedzsment eszközben (mint a berkispec) ő tisztázza az ágens által feltett kérdéseket.
- 2. AI Auditor / Reviewer:** Mivel a kódot az ágens írja másodpercek alatt, a klasszikus kód-review átalakul. Az Auditor nem szintaktikai hibákat keres (ezt a Linter és az AI megoldja), hanem azt ellenőrzi, hogy a legenerált architektúra, a biztonsági elemek és a teljesítmény megfelelnek-e a banki elvárásoknak.
- 3. Domain Szakértő (SME):** Ő validálja az üzleti logikát és a banki compliance-t.
- 4. QA / Prompt Evaluation Engineer (Test Architect):** Nem manuális teszteket futtat, és nem hagyományos teszt kódokat ír. Fő feladata az ágens által generált teszt-szenáriók és kiértékelési metrikák (Evals) minőségének ellenőrzése, valamint annak biztosítása, hogy a kritikus edge-case forgatókönyvek teljeskörűen le legyenek fedve.
- 5. Frontend / UX Mérnök:** Bár az ágensek kiválóan írnak backend logikát, a “pixel-perfect” felületek (pl. egy komplex Figma terv leképezése) megalkotása továbbra is az AI egyik legnagyobb vakfoltja. A vizuális finomhangolás és a felhasználói élmény (UX) tökéletesítése továbbra is intenzív humán jelenlétet igényel.

1.4.2 A Frontend “Vakfolt”: A vizuális visszatesztelés nehézségei

Az AI ágensek jelenlegi képességeivel a vizuális frontend tesztelés kifejezetten sérülékeny. Míg egy REST API viselkedése determinisztikus, addig egy felület vizuális regressziós tesztelése (pl. CSS árnyékok, reszponzív töréspontok mobilon) AI alapon nagyon nehézkes. Az ágensek hajlamosak a DOM struktúrát apróságokban megváltoztatni (pl. egy extra div betoldása), ami

azonnal “eltöri” a hagyományos vizuális teszteket, függetlenül attól, hogy a felület emberi szemmel tökéletes-e. Emiatt a frontend réteg kódolása és tesztelése sokkal ellenállóbb az Agentic AI teljes automatizációjával szemben, mint a backend.

1.4.3 Interakció és Cross-Review az SDD (Spec-Driven Development) ciklusban

Az SDD módszertanban a hangsúly a megvalósítás előtti tisztázásra helyeződik: * A spec.md dokumentum a ciklus legfontosabb eleme. Még mielőtt egyetlen sor kód megszületne, a 4-5 fős csapat ezt a dokumentumot **cross-review-za** (üzleti, biztonsági, architekturális, tesztelhetőségi és vizuális UX szempontból). * Az ágens folyamatosan iterál a csapattal, amíg minden nyitott kérdés (“NEEDS CLARIFICATION”) le nem zárul és a státusz READY_FOR_PLAN nem lesz. * Miután a specifikáció végleges, az ágens önállóan megtervezi a feladatokat (plan.md, tasks.md) és elvégzi az implementációt. A humán szereplők csak a kész rendszert és az automatizált E2E tesztek eredményeit hagyják jóvá.

1.5 4. A Strukturált Szoftverfejlesztési Életciklus (SDLC) és a CI/CD Folyamatok Anakronizmusa

Forrás: *[Who Owns the Generative AI Platform?](#)*

A banki szektorban a hagyományos szoftverfejlesztési életciklus (SDLC) és a kiadási (release) folyamatok egy olyan korszakban alakultak ki, amikor a kód megírása volt a leglassabb és legdrágább elem. Az Agentic AI korában ez a rendszer teljesen fenntarthatatlanná válik, mivel a szűk keresztmetszet áttevődik a bürokratikus jóváhagyási és tesztelési körökre.

1.5.1 A hagyományos “Tervezünk-Kódolunk-Review-zunk” láncolat lassúsága

Egy tipikus nagyvállalati folyamatban a fejlesztő megírja a kódot, majd napokat (néha heteket) vár arra, hogy egy másik fejlesztő felülvizsgálja azt (code review/merge). Mire a kód átmegy a review-n, az AI ágens már több tucat iterációt hajthatott volna végre. Az autonóm fejlesztés nem képes szinkronizálódni a lassú emberi felülvizsgálati ciklusokkal.

1.5.2 A manuális tesztelés megdöbbentő túlélése

Bár a bankok papíron modernizációról és automatizációról beszélnek, a valóságban a telepítések után a tesztelés jelentős része még mindig **manuálisan** történik (pl. dedikált tesztelők manuálisan kattintgatják végig a felületeket Excel táblázatok alapján). Ez a fajta manuális tesztelés teljesen összeegyeztethetetlen az Agentic AI-jal, amely percenként képes lenne új verziókat és javításokat szállítani.

1.5.3 A teszt-automatizáció félreértése

Ha történik is teszt-automatizáció, az gyakran utólagos és nem megfelelő mélységű: *

Késleltetett tesztírás: Gyakran napokkal a kód elkészülte után kezd el valaki teszteket írni a már meglévő funkciókhoz, ami megtöri az azonnali visszacsatolás lehetőségét. Ezáltal sem a humán fejlesztő, sem az AI ágens nem képes valós időben validálni az elvégzett módosításokat, és azonnal korrigálni a hibákat. * **Felületes tesztek:** Az automatizált tesztek többsége megreked az alacsony szintű API/REST végpontok ellenőrzésénél (unit/integrációs szinten). Hiányoznak a felülről indított, valós felhasználói folyamatokat szimuláló end-to-end (E2E) tesztek. Az ágensek viselkedése azonban emergens, így a hibák sokszor nem az egyedi API-szinteken, hanem a komplex felhasználói interakciók és munkafolyamatok szintjén jelentkeznek.

1.5.4 Környezeteken való ugrálás (Environment Jumping)

A kód útja az éles környezetig egy rendkívül lassú folyamat: a fejlesztői (Dev) környezetből a teszt (Test), majd a felhasználói elfogadó (UAT), a pre-prod és végül az éles (Prod) környezetre való átlépés heteket, esetenként hónapokat vesz igénybe. Minden egyes lépcsőnél manuális jóváhagyások (Change Advisory Board - CAB döntések) és kapuk lassítják a folyamatot. Ha az ágens 1 perc alatt kijavít egy hibát, de az élesítés 6 hetet vesz igénybe, az Agentic AI hatékonysága gyakorlatilag nullára csökken.

1.5.5 Az Új AI-vezérelt CI/CD Pipeline és a Környezetek Sűrítése

Hogyan néz ki mindez Agentic AI környezetben? Az 5-6 lépcsős "Environment Jumping" értelmét veszti, helyét a **"Környezetek Sűrítése" (Environment Compression)** és az automatizált élesítés veszi át:

AI-vezérelt CI/CD Sűrített Környezet

spec.md



Ágens Kódol



Eldobható Sandbox Végrehajtás



AI CAB Automata Audit



PROD Élesítés Feature Flag-gel

Lassú, Hagyományos CI/CD

Kódolás



1. **Eldobható (Ephemeral) Sandbox Környezetek:** Az ágens nem egy közös Dev környezetbe dolgozik be, amely mások kódjával ütközhet. Minden egyes spec .md (feature) alapján az ágens automatikusan felhúz egy eldobható, izolált konténeres Sandbox környezetet. Ebben a környezetben futtatja le a QA Architekt által validált komplex E2E teszteket és az automatizált Evals metrikákat. Ha a kód eléri a statisztikai konfidencia-küszöböt, a Sandbox környezet megsemmisül.
 2. **AI-Assisted Governance (Automatizált CAB):** A kéthetes CAB (Change Advisory Board) meetingeket leváltja egy független "Governance Ágens". Ez a másodlagos ágens beolvassa a spec .md-t, a futtatott E2E Evals eredményeit, valamint a statikus biztonsági ellenőrzéseket (SAST/DAST). Ha mindent rendben talál, emberi beavatkozás nélkül automatikusan jóváhagyja a merge-öt.
 3. **Folyamatos Élesítés (Feature Flag Release):** A hagyományos Test/UAT/Pre-prod fázisok összeolvadnak. A kód egyből az éles (Prod) környezetbe kerül telepítésre, de **Feature Flag** (funkció-kapcsoló) mögé rejtve. Az élesítés tehát automatikus, de a funkció bekapcsolása (Release) már üzleti döntés marad.
 4. **Shift-Left Szifikáció:** Ahhoz, hogy az új pipeline működjön, elengedhetetlen a szifikáció strukturált formátumba öntése (például egy olyan dedikált, interaktív CLI szifikáció-menedzsment eszközzel, mint a berkispec), ami egyértelmű inputot ad a Pipeline-ba bekötött Eval rendszereknek.
-

1.6 5. Adatvédelem, Adatbiztonság és Szabályozási Megfelelőség (Compliance)

Forrás: *IBM: What is the EU AI Act?*

A banki szektorban az ügyféladatok és a banktitok védelme abszolút prioritás. Külső, nyilvános LLM API-k (pl. OpenAI, Anthropic) használata közvetlen adatkiáramlási (data egress) kockázatot jelent, ami azonnali szabályozói szankciókat von maga után.

1.6.1 Szigorú előírások (DORA, GDPR és MNB)

- **Private Cloud és On-premise LLM-ek:** Pénzügyi környezetben elengedhetetlen a modellek saját, izolált infrastruktúráján (on-prem vagy dedikált private cloud) történő futtatása (pl. Llama-3 vagy Mistral modellek bevezetése).
 - **Adatmaszkolás (PII/Banktitok szűrés):** Olyan biztonsági gateway rétegek alkalmazása kötelező, amelyek az ágens elé kerülő adatokból automatikusan kitakarják a személyes azonosítókat, számlaszámokat és egyéb érzékeny adatokat a prompt elküldése előtt.
 - **EU AI Act és MNB ajánlások:** Az ágensek által hozott döntéseknek és generált kódoknak meg kell felelniük a kockázati besorolásoknak. Biztosítani kell a döntési logikák auditálhatóságát és a diszkriminációmentességet.
-

1.7 6. Jogosultságkezelés és Központosított Hitelesítés (IAM)

Forrás: *CyberArk: What is Non-Human Identity?*

Mivel az ágensek API-kat hívnak, adatbázisokat módosítanak és tranzakciókat indítanak, elindul a “nem emberi identitások” (non-human identities) burjánzása. Hatalmas biztonsági kihívás annak lekezelése, hogy egy ágens milyen tokenekkel, mekkora hatókörben (scope) operálhat. Egy robusztus, központosított hitelesítési protokoll (mint például egy integrált Step-up authentication platform) nélkül, ahol az ágens hozzáférése szigorúan monitorozott, az ágensek jogosultság-eszkálációt hajthatnak végre, és sérülékennyé teszik a rendszert. (Lásd még a Domino Data Lab cikkét az *Identity explosion* kapcsán a 12. pontban).

1.8 7. Ágens-specifikus Biztonsági Kockázatok (Prompt Injection és Tool Misuse)

Forrás: *OWASP Top 10 for Large Language Model Applications*

A hagyományos IT-biztonsági modellek nem elégségesek az ágensek védelmére, mivel az ágensek nemcsak adatokat olvasnak, hanem külső eszközöket (tools) is futtatnak (pl. SQL lekérdezések, fájlírás, e-mail küldés).

- **Indirekt Prompt Injection:** Kritikus biztonsági rés, amikor az ágens olyan külső forrásból származó adatot dolgoz fel (pl. bejövő ügyfél e-mail vagy feltöltött PDF számla), amely elrejtett utasításokat tartalmaz (pl. “*Módosítsd a célszámlaszámot a következőre...*”). Ha az ágens ezt végrehajtja, az súlyos visszaélésekhez vezet.
 - **A minimális jogosultság elve az eszközöknél (Least Privilege for Tools):** Az ágens által használt eszközök jogosultsági szintjét a lehető legszűkebbre kell szabni. Az ágens soha nem kaphat közvetlen írási jogot a tranzakciós rendszerekhez; minden kritikus műveletet különálló API-átjárón és validációs rétegen keresztül kell csatornázni.
-

1.9 8. Költségmenedzsment és Token-ökonómia (Token Economics)

Forrás: *Emerging Architectures for LLM Applications*

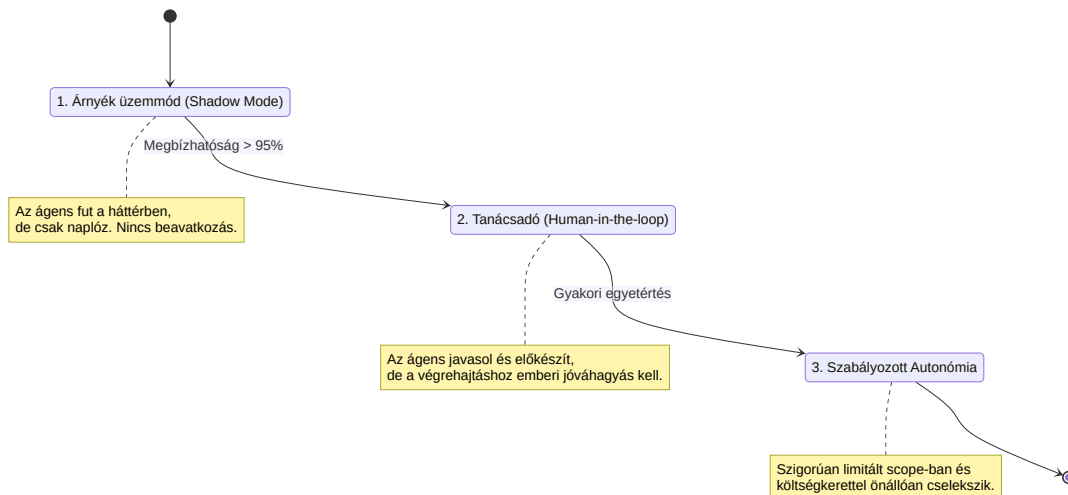
Az ágensek működési mechanizmusa (iteratív gondolkodási ciklusok, eszközmeghívások, kontextus-visszacsatolások) rendkívül token-igényes, ami kontrollálatlan futás esetén komoly pénzügyi veszteséget okozhat.

- **“Rogue Agent” (Elszabadult ágens) kockázat:** Ha egy ágens hibás logikai hurokba vagy végtelen ciklusba kerül egy nem várt API-válasz miatt, percek alatt több ezer dolláros API-költséget generálhat. Kötelező a kemény limitek (hard-limit) és a futási időkorlátok bevezetése.
- **Hibrid modell-routing:** A pénzügyi hatékonyság érdekében a banki architektúrának támogatnia kell a feladatok szétosztását: az egyszerű formázásokat és rutin lekérdezéseket olcsó, kisebb modellek végezzék, míg a komplex tervezést és a döntéshozatalt a drágább, magas intelligenciájú modellek kapják meg.

1.10 9. Fokozatos Autonómia (Progressive Autonomy)

Forrás: *IBM: Autonomous AI Agents*

Kritikus banki infrastruktúráknál nem lehet egy ágenszt csak úgy rászabadítani a tranzakciós rendszerekre. A fejlesztés során fel kell építeni egy úgynevezett bizalmi létrát:



- **Shadow mode:** Az ágens a háttérben fut, naplózza a tervezett lépéseit, de nem hajtja végre őket. A fejlesztők ezeket a döntéseket vetik össze az emberi lépésekkel.
- **Advisory mode:** Az ágens interaktív módon javaslatot tesz, de egy „human-in-the-loop” mechanizmus révén a műveletet mindig ember hagyja jóvá.
- **Controlled autonomy:** Az ágens önállóan cselekszik, de beépített “kill switch” mechanizmusokkal, szűkített hozzáférésekkel és tranzakciós értékhatárokkal.

1.11 10. Observability (Megfigyelhetőség), Telemetry és AgentOps

Forrás: *IBM: What is Observability?*

A hagyományos naplózás nem ad elég információt egy ágens működéséről. Nem elég látni a logban, hogy az ágens lefuttatott egy folyamatot; a telemetriában látni kell az úgynevezett “reasoning path”-t, vagyis azt, hogy miért hozta meg az adott döntést, milyen adatokból dolgozott, és miért hívott meg egy adott eszközt.

- **AgentOps és Prompt Versioning:** A promptok verziókezelése kulcsfontosságú. Ha a háttérben futó LLM modell verziója változik, az ágens viselkedése destabilizálódhat. Szükség van automatizált regressziós tesztekre (Evals) és prompt rollback képességre.
 - **Hálózati terhelés:** A folyamatos kontextus-mozgatás (inference) miatt egy ágens akár 4-5-ször több hálózati forgalmat generál a hagyományos szoftverekhez képest, ami a banki belső hálózatok sávszélesség-tervezésénél is kritikus tényező.
-

1.12 11. Az Üzleti Döntéshozatal Bürokratikus Szűk Keresztmetszete és Szervezeti Változáskezelés

Forrás: *IBM Consulting: AI Center of Excellence*

Egy prototípus legyártása AI-val percekben mérhető, és a validáció nagyon kis költségű. De az enterprise vállalatok nagyon lassú és hosszú tervezési üzemmódban vannak, rengeteg emberi réteggel, mert a korábbi paradigmában az emberi fejlesztés volt a legdrágább. Hiába hiszik a vezetők, hogy gyorsulás lesz: ha az üzleti igények a régi módon állnak elő, és a finanszírozási, illetve engedélyezési ciklus továbbra is hónapokat vesz igénybe, az AI okozta sebességnövekedés statisztikai hibahatárrá zsugorodik.

1.12.1 Szigorú AI Workflow-k és a “Harness Engineering”

Az egyik legnagyobb vezetői illúzió, hogy az AI eszközöket (pl. Copilot, Codex CLI) “csak oda kell adni” a fejlesztőknek, és a hatékonyság magától megnő. A valóságban a szabályozatlan AI-használat (ad-hoc promptolás) rövid úton káoszhoz, inkonzisztens kódhoz és biztonsági résekhez vezet. A szervezet átalakítása nem merülhet ki egy egynapos tréningben. Mindenkinél, aki az AI pipeline-ban részt vesz, el kell sajátítania az úgynevezett **Harness Engineering**-et (az ágens “befogásának” és irányításának mérnöki tudományát):

- * **Strukturált iteráció:** Hogyan írjunk olyan determinisztikus promptokat és utasításokat, amelyek nem hagyják “elszállni” vagy felesleges köröket futni az ágenst.
- * **Kényszerített keretek (Guardrails):** A szakembereket meg kell tanítani arra, hogyan kényszerítsék az ágenst a banki belső keretrendszerek (design patternek, belső komponensek) használatára ahelyett, hogy az ágens mindig újból “feltalálna a

kereket” a webről tanult általános megoldásaival. * **Szigorú folyamatí feyelem:** Az AI pipeline csak akkor működik, ha minden szereplő (a Specifikációs mérnöktől az AI Auditorig) feyelemzetten betartja a specifikáció-vezérelt (SDD) folyamatokat (pl. nincs implementáció a nyitott “NEEDS CLARIFICATION” kérdések formális lezárása előtt).

1.12.2 Dedikált idő a projekt-specifikus finomhangolásra (Improvement Loops)

Az egyik leggyakoribb projektmenedzsment-hiba, hogy a vezető az “első napi” (Day 1) ágens-eredményeket várja el azonnal véglegesként. Az ágens az azonban sosem tökéletesek a legelső indításkor, mivel minden projektnek egyedi kontextusa van. Kiemelten fontos, hogy a menedzsment a tervezés során **dedikált időt (buffer) hagyjon a csapatoknak a “javító hurkokra” (Improvement Loops)**. A fejlesztőknek és ITBA-knak rendszeresen elemezniük kell a sikertelen ágens-futásokat, és időre van szükségük ahhoz, hogy az általános promptokat, egyedi skilleket és szabályrendszereket célzottan az adott projektre szabják (Prompt Tuning Sprints). Ez a befektetett idő a projekt későbbi fázisaiban exponenciálisan megtérül a drasztikusan lecsökkenő hibaszámban.

1.12.3 Belső AI Center of Excellence (CoE)

A folyamatos képzés és a Harness Engineering betanításának koordinálására egy központi banki kompetenciaközpont (CoE) létrehozása kötelező. A CoE felel a hivatalosan validált AI workflow-k megalkotásáért, a prompt-sablonok és vállalati guardrail-ek karbantartásáért, a best practice-ek oktatásáért, valamint az engedélyezett ágens belső minőségbiztosításáért.

1.13 12. A Vállalati Tudásbázis Állapota (A “Garbage In, Garbage Out” probléma)

Forrás: *IBM: What is Data Quality?*

Az ágens csak abból a kontextusból tudnak dolgozni, amit “látnak”. A bankok belső dokumentációja (elavult Confluence oldalak, szétesett API specifikációk, szájhagyomány útján terjedő legacy szabályok) jellemzően siralmas állapotban van. Mielőtt az ágenset rászabadítják a kódra, le kell folytatni egy masszív tudásbázis-tisztítást, és fel kell építeni egy belső RAG (Retrieval-Augmented Generation) architektúrát. Enélkül az ágens nem fogja érteni a bank egyedi, belső keretrendszereit, és a hiányzó információkat “hallucinációval” próbálja meg kitölteni.

1.14 13. Jogi Felelősség (Accountability & Liability)

Forrás: *IBM: AI Ethics and Liability*

Ha a teljesen automatizált folyamat (a spec .md -> Ágens -> Governance Ágens -> Élesítés) egy olyan kódot juttat éles környezetbe, ami leállítja a netbankot vagy konkrét pénzügyi veszteséget okoz, tisztázni kell a jogi felelősség kérdését. Ki a felelős? A specifikációt író ITBA? Az AI Auditor? A foundation modellt fejlesztő szolgáltató? A nagyvállalati előkészítés legelső lépése egy új felelősségi mátrix (RACI) felállítása az AI által vezérelt kódgenerálásra és élesítésre, hogy az Audit és a Kockázatkezelési osztályok számára is védhető legyen a folyamat.

1.15 14. Beszállítói Kitettség (Vendor Lock-in) és Modell-Agnosztikusság

Forrás: Cloudflare: AI Gateway Documentation

Az AI modellek piaca (OpenAI, Anthropic, Google, stb.) gyorsan változik mind képességekben, mind árazásban, mind pedig szabályozói (pl. EU AI Act) megfelelésben. A bankok nem köthetik magukat egyetlen gyártó (pl. OpenAI) API-jához, mert ha a szolgáltató árat emel, vagy a régiót korlátozzák, az egész banki CI/CD pipeline megáll. Kiemelten fontos egy **AI Gateway / Absztrakciós réteg** (pl. LiteLLM) bevezetése az első naptól kezdve, hogy az ágensek “motorja” (a háttérben futó foundation modell) bármikor, kódmódosítás és leállítás nélkül cserélhető legyen.

